Lecture 3 – Bulk RNA-Seq I

Eleana Cabello

Let’s get started!

- Log on to OSCER via Terminal or MobaXterm
- Navigate to your scratch folder: **cd /scratch/USER**
- Then do: **nano copy_lecture_3.sh**

```
#!/bin/bash
#SBATCH --partition=normal
#SBATCH --n-tasks=1
#SBATCH --mem=10GB
#SBATCH --time=01:00:00
#SBATCH --job-name=copy_lecture3
#SBATCH --output=copy_lecture3_stdout.txt
#SBATCH --error=copy_lecture3_stderr.txt
#SBATCH --mail-user=EMAIL@ou.edu
#SBATCH --mail-type=ALL
#SBATCH --chdir=/scratch/USER/

#==============================================================
# BASH Strict mode (i.e. "nag and quit" instead of ignoring some common errors)
set -e       # EXIT the script if any command returns non-zero exit status.
set -E       # Make ERR trapping work inside functions too.
set -u       # Variables must be pre-defined before using them.
set -o pipefail # If a pipe fails, returns the error code for the failed pipe
          #  even if it isn't the last command in a series of pipes.
#==============================================================

cp -r /scratch/elec/lecture_3 .
```

Bulk RNA-seq steps

1. Quality check the FASTQ files
2. Align FASTQ files to a reference
    a. Build reference genome index
    b. Align reads to reference genome
3. Convert SAM files to BAM files
    a. Sort the SAM files - this produces sorted *.bam* files
    b. Index the BAM files - this produces *.bam*.*bai* files
4. Create count matrix from alignments
5. Differential Expression
6. Visualizations

Starting Point

```
-rw-r-----. 1 elec ircfcore 4.1K May 19 14:28 fastq_screen.conf
drwxr-x---. 2 elec ircfcore   64 May 19 14:34 fastq_screen_refs
-rw-r-----. 1 elec ircfcore 8.6K May 19 14:27 get_gene_counts.sh
drwxr-x---. 2 elec ircfcore   12 May 19 14:30 raw_data
drwxr-xr-x. 3 elec ircfcore    4 May 19 14:28 ref_genome
```

RNA-seq pipeline – get_gene_counts.sh

Raw data folder/directory

Reference Genome files – FASTA and GFF

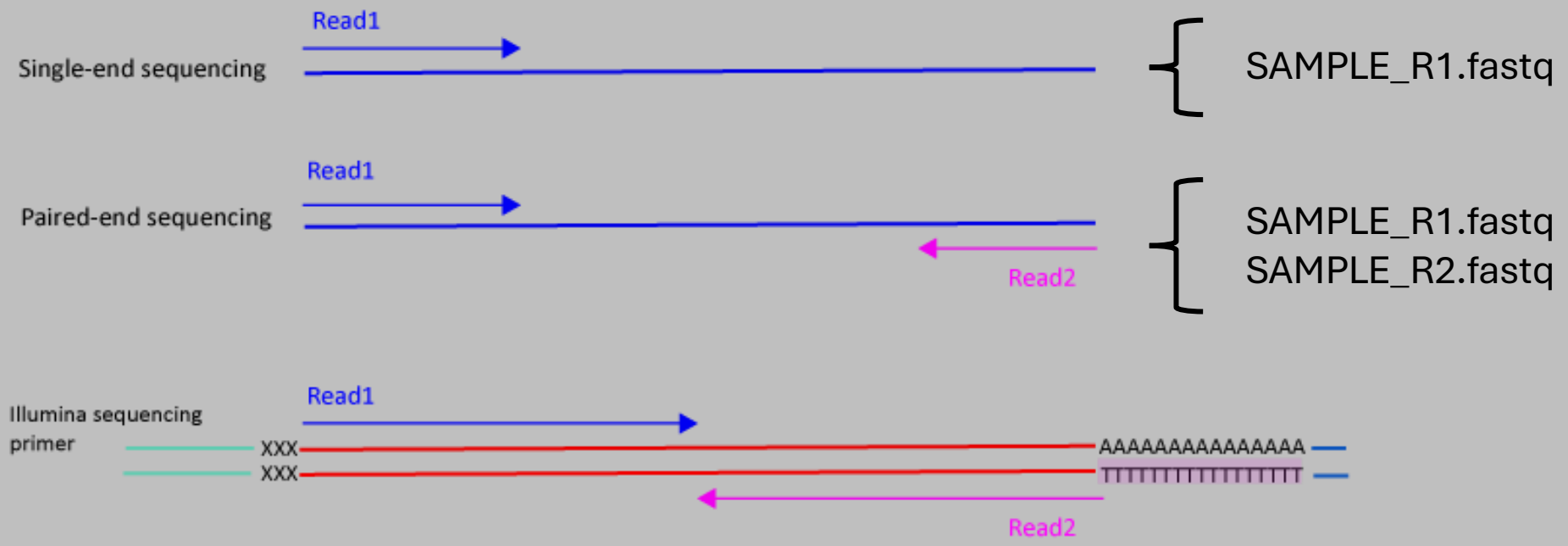
Genomes and configuration file for FastScreen

Let’s go over the data

1. Navigate to /lecture_3/raw_data and look inside.
2. Notice two fastq files per replicate, R1 and R2
3. Look inside file BORED_1_S1_R1_001.fastq.gz using:

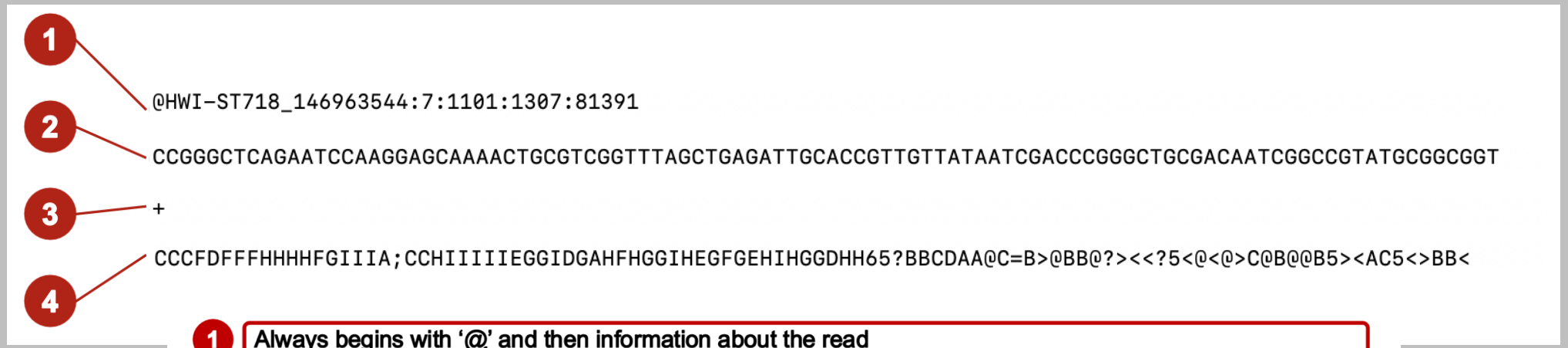
zcat raw_data/BORED_1_S1_R1_001.fastq.gz

Single-end vs Pair-end Reads



Typically for an RNA-Seq projectss you will use Pair-end Reads.

Example FASTQ



1 **Always begins with '@' and then information about the read**

@HWI-ST718_146963544:7:1101:1307:81391

2 **The actual DNA sequence**

CCGGGCTCAGAATCCAAGGAGCAAAACTGCGTCGGTTTAGCTGAGATTGCACCGTTGTTATAATCGACCCGGGCTGCGACAATCGGCCGTATGCGGCGGT

3 **Always begins with a '+' and sometimes the same info in line 1**

+

4 **String of characters representing the quality scores for each nucleotide in Line 2**

CCCFDFFFHHHHFGIIIA;CCHIIIIIEGGIDGAHFHGGIHEGFGEHIHGGDHH65?BBCDAA@C=B>@BB@?><<?5<@<@>C@B@@B5><AC5<>BB<

Phred score

Table 1 ASCII Characters Encoding Q-scores 0–40

Symbol	ASCII Code	Q-Score	Symbol	ASCII Code	Q-Score	Symbol	ASCII Code	Q-Score
!	33	0	/	47	14	=	61	28
"	34	1	0	48	15	>	62	29
#	35	2	1	49	16	?	63	30
$	36	3	2	50	17	@	64	31
%	37	4	3	51	18	A	65	32
&	38	5	4	52	19	B	66	33
'	39	6	5	53	20	C	67	34
(	40	7	6	54	21	D	68	35
)	41	8	7	55	22	E	69	36
*	42	9	8	56	23	F	70	37
+	43	10	9	57	24	G	71	38
,	44	11	:	58	25	H	72	39
-	45	12	;	59	26	I	73	40
.	46	13	<	60	27			

Phred Quality Score	Probability of incorrect call	Base Call Accuracy
10	1 in 10	90
20	1 in 100	99
30	1 in 1000	99.9
40	1 in 10000	99.99

Let’s submit the script!

1. Make sure you are in the directory you want to work in: **/scratch/USERNAME/lecture_3**

2. In the beginning part of the script change YOUR EMAIL LINE and USER: **nano get_gene_counts.sh**

```
#!/bin/bash
#SBATCH --partition=sooner_test
#SBATCH --container=el9hw
#SBATCH --cpus-per-task=10
#SBATCH --mem=30GB
#SBATCH --time=01:00:00
#SBATCH --job-name=get_gene_counts
#SBATCH --output=get_gene_counts_stdout.txt
#SBATCH --error=get_gene_counts_stderr.txt
#SBATCH --mail-user=EMAIL@ou.edu
#SBATCH --mail-type=ALL
#SBATCH --chdir=/scratch/USER/lecture_3
```

Let’s submit the script!

3. Submit the script: **sbatch get_gene_counts.sh**

We will submit the script now, so that it will run while we are going over everything, and we can see the results at the end of class.

4. Let’s make sure your job is running: **squeue --me**

```
#!/bin/bash
#SBATCH --partition=sooner_test
#SBATCH --container=el9hw
#SBATCH --cpus-per-task=10
#SBATCH --mem=30GB
#SBATCH --time=01:00:00
#SBATCH --job-name=get_gene_counts
#SBATCH --output=get_gene_counts_stdout.txt
#SBATCH --error=get_gene_counts_stderr.txt
#SBATCH --mail-user=EMAIL@ou.edu
#SBATCH --mail-type=ALL
#SBATCH --chdir=/scratch/USER/lecture_3

#==============================================================================

# BASH Strict mode (i.e. "nag and quit" instead of ignoring some common errors)
set -e        # EXIT the script if any command returns non-zero exit status.
set -E        # Make ERR trapping work inside functions too.
set -u        # Variables must be pre-defined before using them.
set -o pipefail      # If a pipe fails, returns the error code for the failed pipe
                     #  even if it isn't the last command in a series of pipes.
#==============================================================================
```

Bash Preamble

This section makes the script “Fail Fast”. This means it will stop running as soon as it hits an error at any step.

Load modules we will be using

```
# Load in needed programs as modules
module load FastQC/0.12.1-Java-11      # for Quality Control (QC), analyzing quality of FASTQs
module load FastQ_Screen/0.15.3-GCC-12.3.0 # for QC, comparing data to several references
module load MultiQC/1.28-foss-2024a     # for combining QC reports
module load HISAT2/2.2.1-gompi-2024a      # for aligning to genome, output to .sam files
module load SAMtools/1.19.2-GCC-13.2.0    # for processing/converting/indexing .sam/.bam files
module load Subread/2.0.6-GCC-12.3.0     # for featureCounts, quantifier of read counts per gene
```

Define the file paths to the data we need:

- Reference genome (FASTA file)
- Annotation file (GFF file)
- Reference index folder from Hisat2
- Directory with raw FASTQ reads

```
GENOME_FASTA="ref_genome/genome.fa"
ANNOTATION_FILE="ref_genome/features.gff"
INDEX_GENOME_DIR="ref_genome/hisat2_index”

# Data directory (yes, reading from archive is fine)
FASTQ_DIR="raw_data”
```

```
# Make directories we will use during this analysis
mkdir -p 01-qc
mkdir -p 02-sam
mkdir -p 03-bam
mkdir -p 04-raw_counts
mkdir $INDEX_GENOME_DIR
```

Create directories for each step in the analysis.

This makes it easier to find the files we need.

Try to keep things organized as much as possible!

```
# Step 1 - Quality Control

# fastqc runs a quality analysis for each fastq.gz file

fastqc --threads=$SLURM_CPUS_PER_TASK \
    --outdir 01-qc \
    $FASTQ_DIR/BORED_1_S1_R1_001.fastq.gz \
    $FASTQ_DIR/BORED_1_S1_R2_001.fastq.gz \
    $FASTQ_DIR/BORED_2_S2_R1_001.fastq.gz \
    $FASTQ_DIR/BORED_2_S2_R2_001.fastq.gz \
    $FASTQ_DIR/BORED_3_S3_R1_001.fastq.gz \
    $FASTQ_DIR/BORED_3_S3_R2_001.fastq.gz \
    $FASTQ_DIR/EXCITED_1_S4_R1_001.fastq.gz \
    $FASTQ_DIR/EXCITED_1_S4_R2_001.fastq.gz \
    $FASTQ_DIR/EXCITED_2_S5_R1_001.fastq.gz \
    $FASTQ_DIR/EXCITED_2_S5_R2_001.fastq.gz \
    $FASTQ_DIR/EXCITED_3_S6_R1_001.fastq.gz \
    $FASTQ_DIR/EXCITED_3_S6_R2_001.fastq.gz
```

Here we make sure of the module/program

‘FASTQC’ to check the quality of each FASTQ file.

1. command “fastqc”
2. options “ -- threads= $SLURM_CPUS_PER_TASK”
3. output “ -- outdir 01-qc”
4. input “$FASTQ_DIR/…”

Fastq _screen checks the reads against multiple organisms.

```
# Compare FASTQ files to references
fastq_screen --threads $SLURM_CPUS_PER_TASK --aligner bowtie2 --conf fastq_screen.conf --outdir 01-qc $FASTQ_DIR/BORED_1_S1_R1_001.fastq.gz
fastq_screen --threads $SLURM_CPUS_PER_TASK --aligner bowtie2 --conf fastq_screen.conf --outdir 01-qc $FASTQ_DIR/BORED_1_S1_R2_001.fastq.gz
fastq_screen --threads $SLURM_CPUS_PER_TASK --aligner bowtie2 --conf fastq_screen.conf --outdir 01-qc $FASTQ_DIR/BORED_2_S2_R1_001.fastq.gz
fastq_screen --threads $SLURM_CPUS_PER_TASK --aligner bowtie2 --conf fastq_screen.conf --outdir 01-qc $FASTQ_DIR/BORED_2_S2_R2_001.fastq.gz
fastq_screen --threads $SLURM_CPUS_PER_TASK --aligner bowtie2 --conf fastq_screen.conf --outdir 01-qc $FASTQ_DIR/BORED_3_S3_R1_001.fastq.gz
fastq_screen --threads $SLURM_CPUS_PER_TASK --aligner bowtie2 --conf fastq_screen.conf --outdir 01-qc $FASTQ_DIR/BORED_3_S3_R2_001.fastq.gz
fastq_screen --threads $SLURM_CPUS_PER_TASK --aligner bowtie2 --conf fastq_screen.conf --outdir 01-qc $FASTQ_DIR/EXCITED_1_S4_R1_001.fastq.gz
fastq_screen --threads $SLURM_CPUS_PER_TASK --aligner bowtie2 --conf fastq_screen.conf --outdir 01-qc $FASTQ_DIR/EXCITED_1_S4_R2_001.fastq.gz
fastq_screen --threads $SLURM_CPUS_PER_TASK --aligner bowtie2 --conf fastq_screen.conf --outdir 01-qc $FASTQ_DIR/EXCITED_2_S5_R1_001.fastq.gz
fastq_screen --threads $SLURM_CPUS_PER_TASK --aligner bowtie2 --conf fastq_screen.conf --outdir 01-qc $FASTQ_DIR/EXCITED_2_S5_R2_001.fastq.gz
fastq_screen --threads $SLURM_CPUS_PER_TASK --aligner bowtie2 --conf fastq_screen.conf --outdir 01-qc $FASTQ_DIR/EXCITED_3_S6_R1_001.fastq.gz
fastq_screen --threads $SLURM_CPUS_PER_TASK --aligner bowtie2 --conf fastq_screen.conf --outdir 01-qc $FASTQ_DIR/EXCITED_3_S6_R2_001.fastq.gz

#------------------------------------------------------------------------
# Step 1b (aggregate QC and other info)
# multiqc aggregates the analysis from all the fastqc outputs and other processes.

multiqc . --outdir 01-qc
```

Aggregates all the Fastqc files for each sample

together in a comparable format

```
# Step 2 - Align reads to genome.

# Step 2a. Build index from reference genome
hisat2-build --threads $SLURM_CPUS_PER_TASK $GENOME_FASTA $INDEX_GENOME_DIR/gold_snidget

# Step 2b. Align reads to reference genome
hisat2 --threads $SLURM_CPUS_PER_TASK -x $INDEX_GENOME_DIR/gold_snidget -1 $FASTQ_DIR/BORED_1_S1_R1_001.fastq.gz  -2 $FASTQ_DIR/BORED_1_S1_R2_001.fastq.gz  -S 02-sam/BORED_1.sam
hisat2 --threads $SLURM_CPUS_PER_TASK -x $INDEX_GENOME_DIR/gold_snidget -1 $FASTQ_DIR/BORED_2_S2_R1_001.fastq.gz  -2 $FASTQ_DIR/BORED_2_S2_R2_001.fastq.gz  -S 02-sam/BORED_2.sam
hisat2 --threads $SLURM_CPUS_PER_TASK -x $INDEX_GENOME_DIR/gold_snidget -1 $FASTQ_DIR/BORED_3_S3_R1_001.fastq.gz  -2 $FASTQ_DIR/BORED_3_S3_R2_001.fastq.gz  -S 02-sam/BORED_3.sam
hisat2 --threads $SLURM_CPUS_PER_TASK -x $INDEX_GENOME_DIR/gold_snidget -1 $FASTQ_DIR/EXCITED_1_S4_R1_001.fastq.gz -2 $FASTQ_DIR/EXCITED_1_S4_R2_001.fastq.gz -S 02-sam/EXCITED_1.sam
hisat2 --threads $SLURM_CPUS_PER_TASK -x $INDEX_GENOME_DIR/gold_snidget -1 $FASTQ_DIR/EXCITED_2_S5_R1_001.fastq.gz -2 $FASTQ_DIR/EXCITED_2_S5_R2_001.fastq.gz -S 02-sam/EXCITED_2.sam
hisat2 --threads $SLURM_CPUS_PER_TASK -x $INDEX_GENOME_DIR/gold_snidget -1 $FASTQ_DIR/EXCITED_3_S6_R1_001.fastq.gz -2 $FASTQ_DIR/EXCITED_3_S6_R2_001.fastq.gz -S 02-sam/EXCITED_3.sam
```

Reference genome must be index into smaller files that make it easier to align reads.

Aligning .fastq files reads to reference genome to make .sam files.

```
# Step 3a. - sort your aligned reads. Output is a .bam file.
samtools sort --threads 12 02-sam/BORED_1.sam > 03-bam/BORED_1.bam
samtools sort --threads 12 02-sam/BORED_2.sam > 03-bam/BORED_2.bam
samtools sort --threads 12 02-sam/BORED_3.sam > 03-bam/BORED_3.bam
samtools sort --threads 12 02-sam/EXCITED_1.sam > 03-bam/EXCITED_1.bam
samtools sort --threads 12 02-sam/EXCITED_2.sam > 03-bam/EXCITED_2.bam
samtools sort --threads 12 02-sam/EXCITED_3.sam > 03-bam/EXCITED_3.bam
```

Sorting the .sam files and converting to .bam files

```
# Step 3b. - index the SAM files (needed for some applications)
samtools index 03-bam/BORED_1.bam
samtools index 03-bam/BORED_2.bam
samtools index 03-bam/BORED_3.bam
samtools index 03-bam/EXCITED_1.bam
samtools index 03-bam/EXCITED_2.bam
samtools index 03-bam/EXCITED_3.bam
```

Indexing the .bam files for downstream processes.

```
# Step 4 - Create count matrix

featureCounts \
   -T $SLURM_CPUS_PER_TASK \
   -p \
   -a $ANNOTATION_FILE \
   -o 04-raw_counts/counts.txt \
   03-bam/BORED_1.bam \
   03-bam/BORED_2.bam \
   03-bam/BORED_3.bam \
   03-bam/EXCITED_1.bam \
   03-bam/EXCITED_2.bam \
   03-bam/EXCITED_3.bam
```

Use your genome feature/annotation file to count how many reads mapped to each gene.

Other Aligners and Methods

- Splice-Aware Genome Aligners – STAR, HISAT2
- Transcriptome Aligners (Pseudo/Quasi-Aligners) – Salmon, kallisto
- Transcriptome-Guided Genome Aligners - HISAT2 (with GTF)

Output Checklist

- ❑ FastQC files
- ❑ MultiQC files
- ❑ SAM files
- ❑ Sorted BAM files
- ❑ BAM File in JBROWSE2
- ❑ Count matrix! And Summary file!